

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – SAST SIMULATION

Course Code:	CSA5803	Course Title:	DEVSECOPS ENGINEERING AND APPLICATION SECURITY
CO Assessed:	CO3 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 1
Weightage:	30% of CIA	Total Marks:	100
CO3 Statement:	Simulate Static Application Security Testing (SAST) using synthetic data		
Student Name:	LOKESH KUMAR V	Reg. No.:	192521170
Section / Batch:	C	Date:	13/07/26

Instructions to Students

- Ensure all diagrams and workflow illustrations are **original, clear, and professionally formatted**.
- Include **all editable source files** (such as .yaml, .py, requirements.txt, .pre-commit-config.yaml, and any diagram source files if applicable).
- Submit **both the GitHub repository link** and the **final PDF report** through **Google Classroom** before the specified deadline.
- Verify that the GitHub repository is accessible and contains all required project files, screenshots, and documentation.

Question Type	Focus Area	Questions	Marks
SAST SIMULATION	Creative Things and concept grasping	Q1	Q1 –100
GRAND TOTAL:		100 Marks	

Code of Conduct

I Lokesh Kumarv(192521170) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Lokesh Kumar.v

Signature of the Student:

A BETTER **REAL-TIME SCENARIO** FOR SAST WOULD BE AN **ONLINE BANKING APPLICATION**

Part 1 – SAST Simulation Using a Real-Time Scenario

Scenario: Online Banking Application

1. Real-Time Problem

A bank provides an online banking application where customers can:

- Log in to their accounts
- Check account balance
- Transfer money
- View transaction history
- Update personal information

Before deploying a new version, the development team performs **Static Application Security Testing (SAST)** on the source code to identify security vulnerabilities.

2. Vulnerable Synthetic Code

Python code

```
from flask import Flask, request
```

```
import sqlite3
```

```
app = Flask(__name__)
```

```
@app.route("/transfer")
```

```
def transfer():
```

```
account = request.args.get("account")
```

```
amount = request.args.get("amount")
```

```
conn = sqlite3.connect("bank.db")
```

```
cursor = conn.cursor()
```

```
# Vulnerable SQL query
```

```
query = "SELECT balance FROM accounts WHERE account_no=" + account +  
"""
```

```
cursor.execute(query)
```

```
balance = cursor.fetchone()
```

```
# Transfer operation
```

```
if balance and float(balance[0]) >= float(amount):
```

```
    print("Money transfer successful")
```

```
    return "Transfer successful"
```

```
    return "Insufficient balance"
```

3. SAST Scan

The SAST scanner analyzes the source code without running the application.			
ID	Vulnerability	Severity	Location
SAST-001	SQL Injection	Critical	Line 14
SAST-002	Missing Authentication	High	Line 6
SAST-003	Missing Authorization	High	Line 6
SAST-004	Improper Input Validation	Medium	Lines 9–10
SAST-005	Insecure Transaction Handling	High	Line 20

4. Vulnerability Explanation

SQL Injection – Critical

- Account number is directly inserted into the SQL query.
- The application should use parameterized queries.

Missing Authentication – High

- The transfer endpoint does not verify whether the customer is logged in.
- Authentication should be required before performing transactions.

Missing Authorization – High

- The application does not verify whether the authenticated user owns the account.
- Authorization/RBAC checks should be implemented.

Improper Input Validation – Medium

- account and amount are accepted directly from the request.
- Input should be validated before processing.

Insecure Transaction Handling – High

- A real banking transaction requires secure transaction controls.
- Transactions should use proper database transactions and authorization checks.

5. Secure Code

```
@app.route("/transfer")
```

```
def transfer():
```

```
    account = request.args.get("account")
```

```
    amount = request.args.get("amount")
```

```
    if not account or not amount:
```

```
        return "Invalid input", 400
```

```
    try:
```

```
        amount = float(amount)
```

```
        if amount <= 0:
```

```
            return "Invalid amount", 400
```

```
    except ValueError:
```

```
        return "Invalid amount", 400
```

```
    conn = sqlite3.connect("bank.db")
```

```
    cursor = conn.cursor()
```

```
# Parameterized query
```

```
query = "SELECT balance FROM accounts WHERE account_no=?"
```

```
cursor.execute(query, (account,))
```

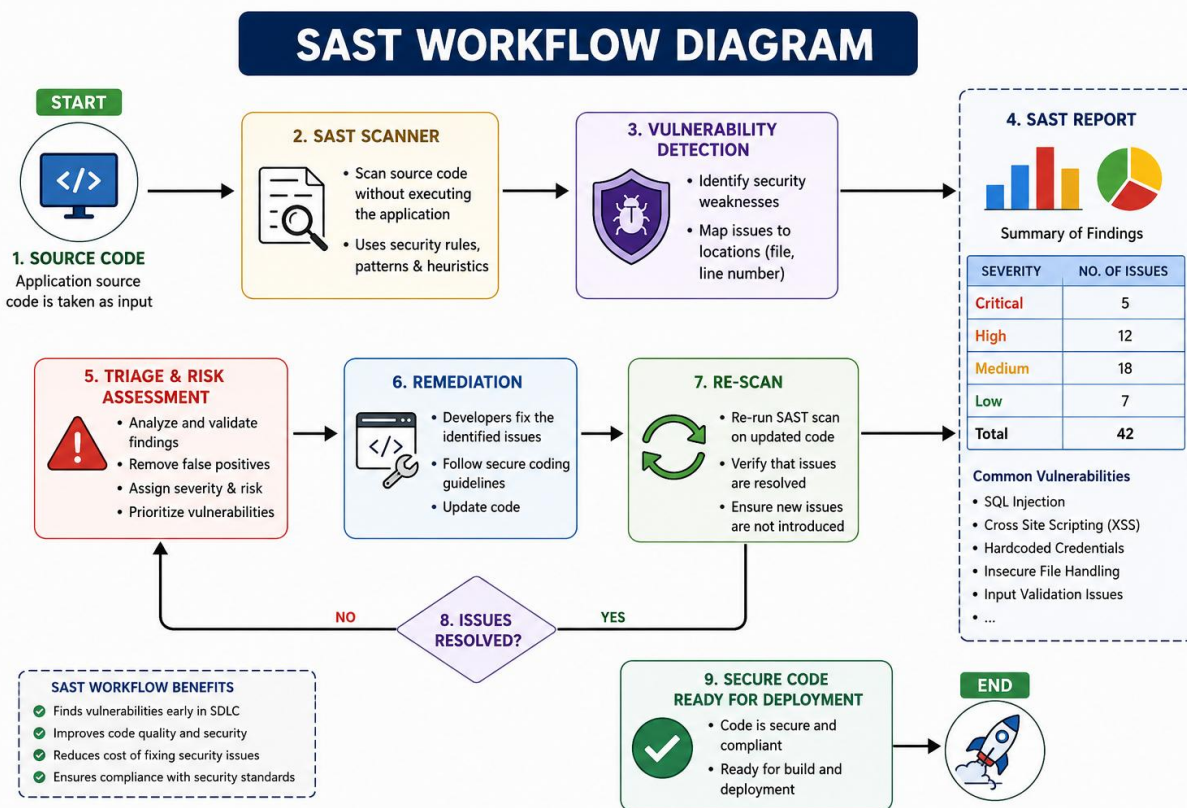
```
balance = cursor.fetchone()
```

```
if balance and balance[0] >= amount:
```

```
    return "Transfer request accepted"
```

```
return "Insufficient balance"
```

6. SAST Workflow



7. Result

The SAST simulation identified **5 security vulnerabilities** in the online banking application:

- **1 Critical**
- **3 High**
- **1 Medium**

After remediation, SQL injection and input-validation issues were addressed, while authentication, authorization, and secure transaction controls were identified as essential security requirements.

8. Conclusion

SAST helps the banking development team identify vulnerabilities **before the application reaches production**. This reduces the risk of unauthorized account access, manipulation of banking data, and insecure transaction processing.